

Verifier-Backed Hard Problem Generation for Mathematical Reasoning

Yuhang Lai^{1,2,*} Jiazhan Feng^{3,*,†} Yee Whye Teh⁴ Ning Miao^{1,2}

¹Department of Data Science, City University of Hong Kong

²Hong Kong Institute of AI for Science, City University of Hong Kong

³School of Intelligence Science and Technology, Peking University

⁴Department of Statistics, University of Oxford

Abstract

Large Language Models (LLMs) demonstrate strong capability in solving scientific and mathematical problems, yet they struggle to produce valid and challenging novel problems—an essential component for advancing LLM training and enabling autonomous scientific research. Existing problem generation approaches either depend on expensive human expert involvement or adopt naive self-play paradigms, which frequently yield invalid problems due to reward hacking. This work introduces VHG, a verifier-enhanced hard problem generation framework built upon three-party self-play. By integrating an independent verifier into the conventional setter-solver duality, our design constrains the setter’s reward to be jointly determined by problem validity (evaluated by the verifier) and difficulty (assessed by the solver). We instantiate two verifier variants: a Hard symbolic verifier and a Soft LLM-based verifier, with evaluations conducted on indefinite integral tasks and general mathematical reasoning tasks. Experimental results show that VHG substantially outperforms all baseline methods by a clear margin.

1 Introduction

Large language model (LLM) have achieved expert-level ability in solving scientific problems. For example, OpenAI’s o1 surpassed PhD-expert baselines on GPQA-Diamond [21, 18], while AlphaGeometry [25] and AlphaProof [10] demonstrated olympiad-level mathematical reasoning, making them very useful in doing research under human supervision. In scientific research, raising a meaningful new question is at least as critical as solving an existing one. This raises a natural question, Can LLMs truly generate valid new questions, enabling them to complete the research cycle and achieve fully autonomous research?

Generating valid new questions is not only pivotal to scientific research but also to the training of LLMs themselves. For example, Gao et al. [4] found that data difficulty is one of the primary factors influencing the performance of LLMs after post-training. However, current post-training paradigms mostly rely on static human-written datasets (e.g., MATH [7]), or offline transformations and post-training recipes (e.g., MetaMath [27], WizardMath [15], ToRA [5], DeepSeekMath [22], and R-Zero [8]). To continuously increase the difficulty of training data, we need to hire domain experts either to compose new problems or modify existing ones, which is not only slow and costly, but also limits LLMs’ ability to the human level.

Numerous previous efforts have been made on generating difficult problems. For example, PromptCoT [31], CHASE [19], and MathSmith [29] use human-designed scaffolds, independently verifiable

*Equal contribution.

†Work done during Jiazhan’s visiting at the Department of Statistics, University of Oxford.

components, or predefined difficulty strategies to build new problems. MathFusion combines old problems through sequential, parallel, and conditional fusion strategies [20]. Though effective in certain tasks, these methods are still upper-bounded by human design. Self-play based methods almost eliminate reliance on human expertise. They use a setter LLM to propose new problems, and another solver LLM to measure the difficulty of the proposed problems. Then, the negative problem accuracy is used as a reward for the setter, so that it will learn to generate harder and harder problems. Though looks elegant, vanilla self-play has a critical flaw, that is the proxy reward of problem difficulty can be easily hacked by generating invalid problems, where the solver has zero accuracy, which provides high rewards to the setter.

In this paper, we address this problem by adding an additional verifier to the two-party game in self-play. Specifically, we propose *VHG: verifier-backed hard problem generation*: a setter proposes both the problem and solution as a pair (x, y^*) , and a verifier accepts or rejects the pair based on its correctness. This design achieves two key goals: On the solver side, it eliminates the noisy training signals introduced by invalid problem-reference pairs. More importantly, on the setter side, for a generated problem, a high reward (i.e., low solver accuracy) can only result from the problem’s true difficulty, making it impossible for the setter to hack the reward system by generating invalid problems.

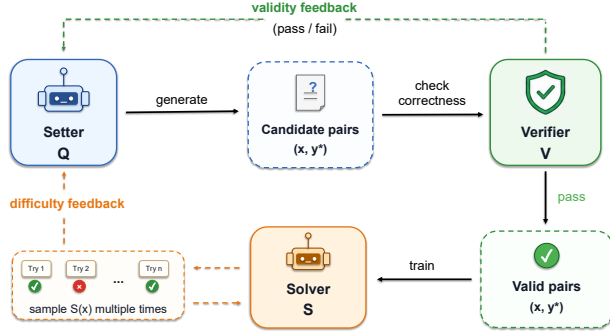


Figure 1: VHG framework. The setter proposes problem-reference pairs, the verifier gates validity, and accepted pairs are scored by solver difficulty for training and challenge construction.

We explore two types of verifiers: Hard and Soft. Hard verifiers leverage symbolic verification mechanisms to provide nearly 100% reliable verifications. Soft verifiers use LLMs to check the correctness of the step-by-step problem generation process. Though less accurate than hard verifiers, soft verifiers enables the framework to work on broader domains. To validate the concept of VHG, we first focus on the task of indefinite integral, a self-contained environment, allowing us to examine the inner working mechanisms of VHG. We also test VHG with a soft verifier in the general math domain, where exact verification is impractical, demonstrating its generalization potential.

Our experiments show that VHG produces problem-reference pairs that are both valid and challenging, proving useful for both solver training and challenge dataset construction. For indefinite integral, VHG improves pass@1 accuracy on AntiderivBench Qualifier/Competition and Integration Stress Test by 16.9%, 16.6%, and 21.4%, respectively, significantly outperforming baseline approaches, including R-Zero, which is the current state-of-the-art model. For general math, VHG enhances performance across a wide range of benchmarks including MATH, AMC, Minerva, Olympiad, and AIME24-26: raising overall pass@1 accuracy from 56.8% to 69.0% and also outperforming baselines by a substantial margin. Beyond performance gains, we find that even though both the setter and the solver are based on Qwen3-4B, VHG can generate problems that challenge larger models (Qwen3-8B, 14B, and 32B), indicating that stronger models can benefit from training with data generated by weaker models.

In summary, the contributions of this paper are as follows:

- We design a novel three-party self-play framework for generating challenging problem, incorporating an additional verifier module. This framework avoids the generation of invalid problems, which would otherwise lead to a collapse of the training pipeline.
- We conduct an extensive empirical study of VHG using both hard and soft verifiers, demonstrating that VHG can generate novel, challenging problems that in turn boost performance across multiple benchmarks.
- We perform a comprehensive analysis of VHG’s working mechanism, highlighting the critical role of the verifier in ensuring the quality of generated problems.

2 Method: The VHG Framework

Verifier-backed hard problem generation (VHG) is a three-party self-play framework designed to generate mathematical problems that are valid, novel, and challenging. It extends the standard setter–solver self-play paradigm by introducing a verifier: the setter proposes a problem-reference pair, the solver provides feedback on its empirical difficulty, and the verifier validates the pair’s correctness. During training, generated pairs are first checked for validity, and only verifier-accepted pairs are scored for difficulty or used to train the solver. This ensures that invalid or underspecified pairs are excluded from downstream processes, mitigating reward hacking and ensuring the quality of generated problems.

Setter Q. The setter Q is responsible for generating problem-reference pairs (x, y^*) , where x denotes a proposed mathematical problem and y^* is its reference solution. This extends the conventional self-play formulation, where the setter typically only needs to propose a problem without a reference solution. In our framework, the reference solution y^* is also part of the generated output. This inclusion enables verifiable validity checks and provides a target against which solver outputs can be evaluated. The setter’s role is architecture-agnostic: it can modify existing problems, compose multiple seed problems, or synthesize entirely new problems from broader contextual information. In this work, we instantiate Q as an LLM conditioned on seed problem-reference pairs. Unlike fixed hand-designed problem generators, such as template libraries or rule-based perturbation systems, our LLM-based setter is trained with feedback from both the solver and the verifier. The training signal incentivizes the setter to generate problems that are not only difficult for the solver but also formally valid according to the verifier’s criteria.

Solver S. The solver S is another model whose failures reflect the difficulty of problems. Given a generated problem x , the solver produces solution attempts under a fixed sampling budget, and its empirical accuracy $\text{Acc}_S(x, y^*)$, defined as the proportion of correct solutions, serves as the difficulty signal in the VHG framework. This role aligns with the solver’s function in standard reinforcement-learning or self-play settings: it learns to solve problems, while its current limitations reveal which problems are sufficiently challenging. A key distinction in VHG is that solver failure only contributes to the setter’s training signal if the verifier first accepts the corresponding problem-reference pair. This decouples validity from difficulty, ensuring that the setter is not rewarded for generating invalid or underspecified problems that the solver cannot answer.

Verifier V. The verifier V validates the generated pair (x, y^*) by checking whether they meet a pre-defined acceptance criterion. In this work, we instantiate two distinct types of verifiers—hard verifiers and soft verifiers to accommodate different task constraints and availability of formal validation tools. Hard verifiers are usually external computational checks to provide definitive validity judgments. For example, in the context of indefinite integrals, we use SymPy (a symbolic mathematics library) to verify that the proposed antiderivative correctly differentiates to the integrand. Hard verifiers offer high auditability and correctness guarantees when such formal checks are feasible. In contrast, soft verifiers use LLMs to validate problem-reference pairs. While soft verifiers may introduce noise compared with hard verifiers, they provide a generalizable solution that can be applied to a broader range of mathematical tasks where formal symbolic checks are unavailable or impractical.

Training. The introduction of the verifier addresses a critical limitation of vanilla self-play: without a validity check, the setter can exploit reward signals by generating invalid or underspecified problems that the solver fails to answer, leading to spurious difficulty. In VHG, the setter’s reward is conditioned on verifier acceptance, ensuring that solver failure only contributes to the reward if the problem-reference pair is valid. Formally, the setter’s reward function is defined as:

$$R_Q(x, y^*) = \mathbf{1}_{[V(x, y^*)=1]} (1 - \text{Acc}_S(x, y^*)), \quad (1)$$

where $\text{Acc}_S(x, y^*)$ is estimated under a fixed sampling protocol. If the verifier rejects the pair (when $V(x, y^*) = 0$), the solver’s failure does not contribute to the reward. This design explicitly separates validity from difficulty, preventing invalid problem-reference pairs from being misclassified as hard examples.

This differs from consensus-backed reward mechanisms, in which the reference answer is determined via majority voting across multiple solver runs, and problem validity is indirectly gauged by the

degree of agreement among these outputs. In our framework (Eq. 1), solver behavior only influences the setter’s reward after the verifier has confirmed validity. The practical VHG pipeline follows a consistent high-level workflow across tasks: (1) collect cold supervised fine-tuning (SFT) data; (2) initialize the setter via cold SFT; (3) train the setter using verifier-backed reward; (4) sample generated problem pools; (5) apply verifier and data-quality filters; (6) score accepted pairs by solver difficulty; and (7) use selected pairs for challenge evaluation or solver training.

For solver training, the verifier gate is applied at the data level. Let $\mathcal{D}_V = \{(x, y^*) : V(x, y^*) = 1\}$ denote the verifier-accepted training pool. For each $(x, y^*) \in \mathcal{D}_V$, the solver receives the usual task-correctness reward, written at the problem level as

$$R_S(x, y^*) = \text{Acc}_S(x, y^*), \quad (x, y^*) \in \mathcal{D}_V. \quad (2)$$

Compared with training on unverifiable synthetic pairs, this data-level verifier gate avoids introducing invalid problem-reference pairs into the solver’s reinforcement-learning signal.

The resulting procedure alternates between setter generation, verifier acceptance, solver-based difficulty scoring on accepted pairs, and downstream use of verifier-accepted pools. Algorithm 1 in Appendix A gives the full pseudocode. In Sections 3.1 and 3.2, we instantiate the verifier V with a SymPy exact checker and an LLM-as-a-judge verifier, respectively.

3 Instantiations of VHG with Hard and Soft Verifiers

In this section, we introduce in detail how to apply the VHG framework to generate difficult and valid problems with two exemplar tasks. For VHG with a hard verifier, we choose the task of indefinite integral, which provides a self-contained testbed where symbolic verification is available. We then apply VHG with a soft verifier to general math to demonstrate the broader use of VHG.

3.1 Hard Verifier on Indefinite Integral

Indefinite integral provides a perfect testbed for VHG with hard verifiers. The problem-reference pair (x, y^*) becomes (f, F) , where f and F are the *integrand* and the corresponding *antiderivative*. This form allows us to strictly check the validity of any generated pair (f, F) . In practice, we use SymPy to first validate the format of both f and F . Then we take the derivative F' of F , and check whether F' equals f . The hard verifier for indefinite integral can be formulated as

$$V_{\text{int}}(f, F) = \mathbf{1}_{[(f, F) \in \mathcal{A}_{\text{format}} \cap \mathcal{A}_{\text{match}}]}, \quad (3)$$

where $\mathcal{A}_{\text{format}}$ denotes the set of well-formed formulas and $\mathcal{A}_{\text{match}}$ denotes the set of (f, F) pairs with matched F' and f .

3.2 Soft Verifier on General Math

We extend the VHG framework to general math tasks by replacing hard verifiers with soft ones. Specifically, for a generated problem-reference pair (x, y^*) , we use LLMs to verify its validity. General math is more open-ended than indefinite integral and therefore more exposed to reward hacking through malformed, underspecified, or superficially hard generations [11, 6]. We use explicit validation rules to make the judge’s decision better grounded: before judge evaluation, a hard-coded filter rejects malformed outputs, missing or multiple final answers, trivial copies, and other failures that do not require model judgment. The LLM judge then checks the validity of both the problem and the answer, as well as their correspondence. The verifier can be formulated as

$$V_{\text{math}}(x, y^*) = \mathbf{1}_{[(x, y^*) \in \mathcal{A}_{\text{filter}} \cap \mathcal{A}_{\text{LLM}}]}. \quad (4)$$

where $\mathcal{A}_{\text{filter}}$ denotes the hard-coded filter and $\mathcal{A}_{\text{LLM}}$ denotes the LLM judge. We show our prompts for the soft verifier in Appendix E.4; the rubric is task-specific and can be extended with additional checks when stronger validation criteria are available.

For both hard and soft verifier settings, to make sure setter Q generates a diverse set of problems, we use existing datasets as seed sets and randomly feed seed problems as hints for Q to generate harder ones. We initialize the setter Q by supervised finetuning on a small group of examples of problem generation. The details can be found in Appendix A.

Table 1: Representative seed-to-generated examples. The examples illustrate structural changes made by the setter in each task.

Task	Original seed	Generated problem	Transformation illustrated
Indefinite integral	Seed function $-1/x$	$\int \frac{\sin(\ln x) - \cos(\ln x)}{x^2} dx$	Product-style modification hides a reciprocal derivative inside a logarithmic oscillatory term.
Indefinite integral	Seed function $x - \arctan(x)$	$\int \frac{-x^2 + 2x \arctan(x)}{(1 + x^2)^2} dx$	Rational wrapping turns a simple inverse-trigonometric seed into a quotient-rule problem.
Indefinite integral	Seed function $e^x - 2\sqrt{x}$	$\int \frac{e^x(2x + 1) - 1}{2\sqrt{x}} dx$	Multiplication by $\sqrt{x}$ combines exponential and radical terms in the final integrand.
General math	How many positive divisors does $6!$ have?	How many positive divisors of $10!$ are perfect squares?	Adds a square-divisor constraint, requiring parity reasoning over prime exponents.
General math	What is the base-2 representation of 84_{10} ?	Convert the base-5 number 123_5 to base 2.	Changes direct base conversion into a two-stage conversion through decimal form.
General math	Fully factor $2x^2 - 8$.	Fully factor $12x^4 - 27$.	Moves from a quadratic common-factor pattern to a quartic difference-of-squares structure.

4 Experiments

We now evaluate VHG in both hard and soft verifier settings in two research questions:

- **RQ-1:** Can VHG generate more difficult yet valid problems as expected?
- **RQ-2:** Can the generated problems lead to a stronger solver after RL training?

Experimental setup For both settings, we use Qwen3-4B-Base as the backbone model for the setter Q and solver S . For indefinite integral, we collect a small number of problems from a college textbook and build a moderate-level difficulty seed set from it for setter training. For general math, we sample a seed pool from MATH dataset [7] and select an easy subset with Qwen3-4B-Base pass@1 at least 0.75 for setter RL. To evaluate the performance of the solver, we compare VHG with several baselines, including vanilla GRPO and R-Zero, on a wide range of benchmarks, including AntiderivBench and our curated Integral Stress Test for indefinite integral, and MATH [7], GSM8K [2], AMC, Minerva, Olympiad, AIME 2024, AIME 2025, and AIME 2026 for general math. A context window of 8192 tokens is used for the most challenging AMC and AIME benchmarks, and 4096 tokens is used for all other benchmarks. We report performance using pass@1 and pass@8, computed under a fixed sampling budget for each problem. Indefinite-integral evaluations use 64 samples per problem, while general math evaluations use 16 samples per problem.

4.1 RQ-1: Evaluation of the Generated Problems

Before checking the end-to-end performance improvement of the solver, we first check whether VHG successfully generates novel valid problems that are challenging to current LLMs.

We use the trained setter Q to generate problem-reference pairs and retain only those accepted by the verifier V . Figure 2 compares the local-solver pass-rate distribution of the setter-RL seed data with the verifier-valid generated pool after setter RL. The generated distribution is normalized over accepted or audit-valid generated items rather than over all raw generations. In both settings, the valid generated pool contains verifier-valid items in lower pass-rate bins that are absent or less represented in the seed data, indicating that VHG changes the difficulty profile while retaining verifier-valid problem-reference pairs. After sampling the most difficult problems with zero pass rate, we find that these problems remain challenging even for stronger models which are up to 8 times larger. Table 2 shows that Pass@1 remains below 50%, with 14% and 30% of the problems unsolved under Pass@8 for indefinite integral and general math, respectively. This indicates that VHG can not only generate harder problems for the comparable solver, but also provide valuable and challenging data for more powerful models, which sheds light on a scalable path to weak-to-strong data generation.

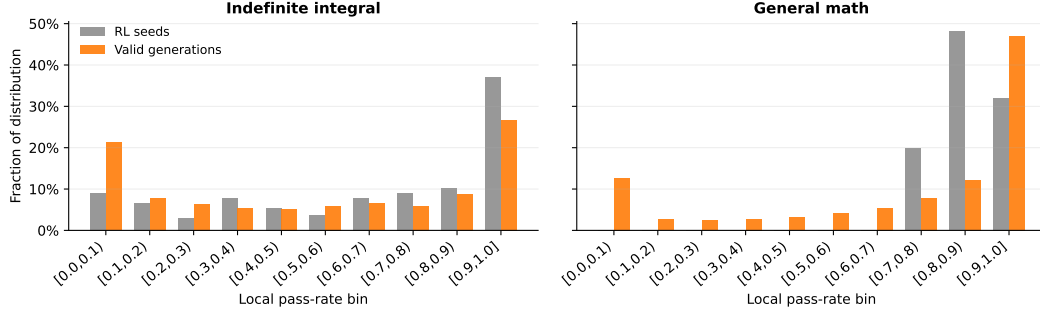


Figure 2: Difficulty distributions of seed problems and verifier-valid VHG generations. Lower Pass@1 bins indicate harder problems.

Table 2: Stronger-solver measurements on verifier-accepted challenge pools. Integration uses 64 samples per problem; general math uses 16. Values are percentages.

Task	Qwen3-8B		Qwen3-14B		Qwen3-32B	
	Pass@1	Pass@8	Pass@1	Pass@8	Pass@1	Pass@8
Indefinite integral	39.48	78.98	49.23	85.51	46.99	85.99
General math	34.50	59.06	41.50	67.94	41.31	70.71

4.2 RQ-2: End-to-End Performance Evaluation of the Solver

We already know that VHG can generate a diverse pool of harder valid problems, but by how much they can boost the training of the solver remains a question. In this part, we answer this question by comparing the solver trained with data from VHG with baseline methods.

Table 3: Integral solver-training comparison and ablation. Values are percentages. R-Zero is the consensus baseline; Seed-only and Seed + Cold-SFT are training-data ablations; Exact-Verified uses hard-verifier accepted generated data.

Method	Comp. Pass@1	Comp. Pass@8	Qual. Pass@1	Qual. Pass@8	Stress Test Pass@1	Stress Test Pass@8
Qwen3-4B-Base	28.8	57.0	52.5	80.0	43.3	69.8
Vanilla GRPO	38.8	61.2	66.5	80.3	60.3	75.4
w. SFT data	43.0	63.9	66.4	83.0	57.5	77.2
R-Zero						
Iter. 1	31.9	58.9	62.7	82.0	51.0	71.3
Iter. 2	30.7	56.3	62.8	81.6	52.9	71.7
Iter. 3	30.5	55.5	62.0	80.7	51.0	70.2
VHG (Hard)	45.4	66.5	69.4	83.1	64.7	81.7

Indefinite integral. As shown in Table 3, VHG raises Pass@1 from 28.8%, 52.5%, and 43.3% to 45.4%, 69.4%, and 64.7% on Competition, Qualifier, and the Integral Stress Test, respectively. In comparison, R-Zero does not outperform vanilla GRPO training on the existing dataset after three iterations, which shows the importance of hard verification in self-play. We also observe that VHG significantly improves Pass@8 on all benchmarks, indicating that the generated problems can further push the capability boundary of the trained LLM.

General math. From Figure 3, we can see that VHG outperforms R-Zero on all benchmarks except GSM8K. This is expected because VHG aims to generate harder problems, which leads to a distribution shift from the primary-school-level problems in GSM8K. The results indicate that a soft verifier is still beneficial in scenarios where strict verification is impossible. Appendix C gives the full subgroup and ablation table.

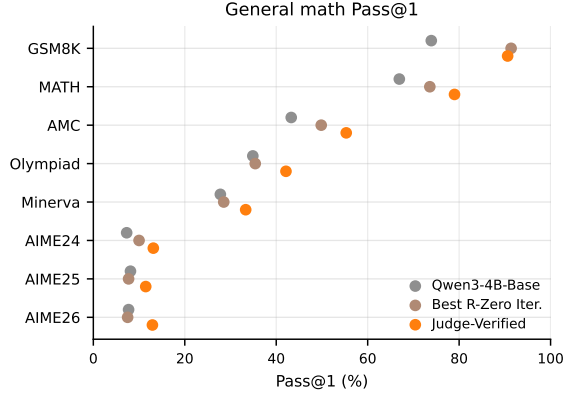


Figure 3: General math Pass@1 profile. Values are percentages under the standardized evaluation suite.

5 Analysis on the Source of Improved Problem Generation Quality

We next analyze why VHG improves problem generation. We use two complementary views: the setter’s training trajectory and the distributional characteristics of generated problems from VHG and R-Zero. Appendix D gives additional curation, novelty, and general-math distribution details.

5.1 Setter Learning Dynamics

Figure 4 shows a two-phase pattern in the hard-verifier setter: validity is learned first, and difficulty follows. From step 0 to step 50, the reference-valid rate rises from 30.6% to 65.2%, while the solver pass rate among valid samples slightly increases from 36.2% to 42.0%. Thus, the early improvement is primarily a validity improvement, not yet a difficulty improvement. After validity is partially established, the generated samples become substantially harder: from step 50 to step 200, the solver pass rate among valid samples falls to 17.6%, while the reference-valid rate ends at 75.5% after only a small intermediate decrease, and the share of all validation samples that are both valid and hard, defined here as local pass rate at most 0.3, rises from 27.5% to 58.5%. The rollout-window summaries show the same direction. This trajectory is consistent with the mechanism enabled by verifier-backed reward: once the verifier makes generated problem-reference pairs trustworthy enough to score, solver feedback can push the setter toward harder valid problems.

5.2 Generation Distribution Analysis

We next ask whether local solver difficulty alone is sufficient to identify useful generated data. For each indefinite-integral problem-reference pair (x, y^*) , we estimate difficulty by the accuracy of Qwen3-4B-Base over 10 samples. Because this task has a hard verifier, we can measure validity directly. As shown in Figure 5(Bottom), R-Zero shifts toward easy-to-intermediate problems as training proceeds. Under its consensus construction, the pseudo-label must have at least one supporting answer, which leaves no retained candidates below 0.1 accuracy. On the contrary, a large proportion of generated problems from VHG have accuracy below 0.2.

Figure 5 provides a mechanical decomposition of this observation. We can see that VHG not only generates more hard problems (46.0% in the $[0.0, 0.1)$ bin), but also has a higher validity rate. For each model, the validity rates for problems decrease almost monotonically as difficulty increases. The reason is that as problems get more complicated, there is a higher chance to make mistakes in generating them. Even so, VHG keeps a relatively high validity rate for the hardest problems, resulting in significantly more challenging problems in the filtered problem set.

We observe a similar phenomenon (see Figure 9) on the task of general math with the soft verifier. This further supports the importance of having a separate verifier module in self-play. With its validity feedback, the setter learns to focus more on the correctness of hard problems. Together with

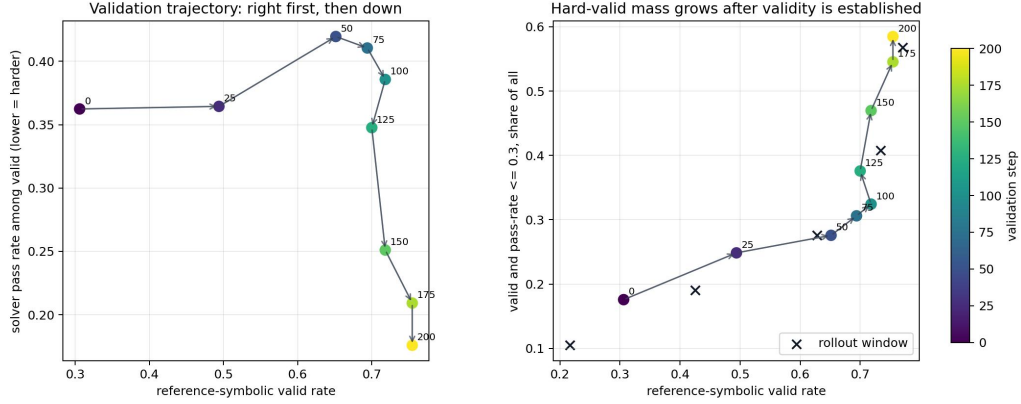


Figure 4: Learning trajectory of the hard-verifier setter on indefinite integral. Validity improves first; later, solver pass rate decreases while the valid-and-hard fraction rises. Lower solver pass rate indicates harder generated problems.

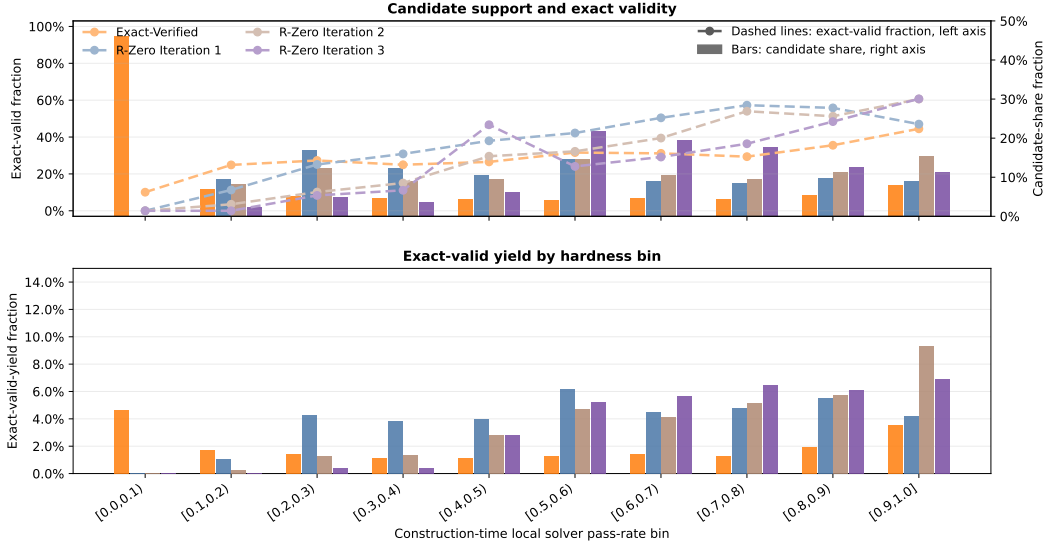


Figure 5: Hardness-validity bins for indefinite integral. Bars show candidate share (top) and exact-valid yield (bottom); dashed curves show exact-valid fraction. Lower pass-rate bins are harder.

difficulty feedback, it further promotes the setter to increase the proportion of hard problems for higher rewards.

6 Related Work

Synthetic mathematical data. A large body of recent mathematical reasoning work improves solvers by expanding the supervision available around human-provided seed problems. MetaMath [27] rewrites seed problems from multiple perspectives, WizardMath [15] adapts instruction evolution and reinforcement feedback to mathematics, and MathScale [23] and KPMath [9] mine topic, concept, or key-point structure from existing problems to synthesize larger training corpora. OpenMathInstruct-2 [24] and the AIMO-2 [17] recipe scale this direction further through generated solution traces, tool-integrated reasoning, filtering, and selection. These systems demonstrate that generated mathematical

supervision can substantially improve solvers. They mostly treat the problem source as fixed or seed-driven, however, and focus on producing more diverse problems, solutions, or traces. In this paper, we make the problem-reference pair itself the object generated by the policy, so the central question shifts from how to scale supervision to when a generated hard pair is valid enough to trust.

Hard and verifiable problem generation. A closer line of work explicitly targets difficult or checkable generated problems. Li et al. [12] abstracts problems into algorithms, implements executable functions, and contextualizes them into new word problems that can be checked by those functions. CHASE [19] generates challenging evaluation items bottom-up from simpler, independently verifiable subtasks. PromptCoT [31] uses construction rationales to synthesize Olympiad-level problems, while SHARP [26] and SAND-Math [16] add generation, filtering, and difficulty-oriented steps to obtain useful reasoning problems for RL or fine-tuning. MathSmith [29] goes further toward adaptive generation by training a reinforced policy to forge harder problems under structural and answer-consistency constraints. Our VHG complements these efforts: it also seeks hard synthetic mathematical data, but it makes the verifier part of the setter’s reward. A candidate receives a difficulty reward only after the generated problem-reference pair has been accepted, so invalid hard generations are removed from valid difficult problems, before solver failure is used as evidence of difficulty.

Self-play and verifier-backed reward. Self-evolving training has also been studied beyond math problem synthesis. SPIN [1] uses a self-play objective to improve a model from its own generations. Yuan et al. [28] uses the model as a judge to create iterative preference data, and DeepSeek-R1 [3] shows the strength of reinforcement learning with verifiable rewards over externally supplied prompts. The closest systems introduce generated tasks into the training loop. Absolute Zero [30] lets a model propose and solve tasks, using a code executor to validate task proposals and answers. R-Zero [8] co-evolves a challenger and solver from zero external data; the challenger proposes problems near the solver frontier, and the solver learns from generated problem-answer data supported by consensus or pseudo-labeling. Weakness-driven and variation-based methods such as SwS [13] and SvS [14] similarly synthesize new problems from the model’s current failures or correct solutions to sustain RLVR training.

Our work is closest in spirit to these setter–solver systems, especially Absolute Zero and R-Zero, but differs in where trust enters the loop. Consensus, answer extraction, and model agreement can provide useful pseudo-labels, yet they are indirect evidence of validity when the problem itself is generated. This is precisely the setting where proxy rewards can be exploited: recent work on reward hacking and verifier gaming shows that models may optimize what a verifier or reward proxy fails to enforce [6, 11]. VHG therefore makes validity a first-class gate in the reward oracle: solver failure contributes to setter reward only after a hard verifier or soft judge verifier accepts the generated problem-reference pair.

7 Conclusion

Hard synthetic mathematical problems should be studied through the reward oracle that selects them, not only through the generator that proposes them. Across hard-verifier integration and soft-verifier general math, VHG checks validity before interpreting solver failure as difficulty. The exact integration setting provides the cleanest evidence: verifier-accepted generations remain difficult for stronger solvers, improve a downstream 4B solver, and expose validity gaps in consensus-generated pseudo-label pools. General math shows that the same ordering remains useful with noisier LLM-as-a-judge validation. Overall, verifier quality determines how far hard-problem generation can be pushed.

Acknowledgments

We would like to thank Xingyu Shen for his support on this work.

References

- [1] Zixiang Chen, Yihe Deng, Huizhuo Yuan, Kaixuan Ji, and Quanquan Gu. Self-play fine-tuning converts weak language models to strong language models. In Ruslan Salakhutdinov, Zico

- Kolter, Katherine Heller, Adrian Weller, Nuria Oliver, Jonathan Scarlett, and Felix Berkenkamp, editors, *Proceedings of the 41st International Conference on Machine Learning*, volume 235 of *Proceedings of Machine Learning Research*, pages 6621–6642. PMLR, 21–27 Jul 2024. URL <https://proceedings.mlr.press/v235/chen24j.html>.
- [2] Karl Cobbe, Vineet Kosaraju, Mohammad Bavarian, Mark Chen, Heewoo Jun, Lukas Kaiser, Matthias Plappert, Jerry Tworek, Jacob Hilton, Reiichiro Nakano, Christopher Hesse, and John Schulman. Training verifiers to solve math word problems. *arXiv preprint arXiv:2110.14168*, 2021.
 - [3] DeepSeek-AI, Daya Guo, Dejian Yang, Haowei Zhang, Junxiao Song, Ruoyu Zhang, Runxin Xu, Qihao Zhu, Shiron Ma, Peiyi Wang, Xiao Bi, et al. Deepseek-r1: Incentivizing reasoning capability in llms via reinforcement learning, 2025. URL <https://arxiv.org/abs/2501.12948>.
 - [4] Zhaolin Gao, Joongwon Kim, Wen Sun, Thorsten Joachims, Sid Wang, Richard Yuanzhe Pang, and Liang Tan. Prompt curriculum learning for efficient LLM post-training. In *The Fourteenth International Conference on Learning Representations*, 2026. URL <https://openreview.net/forum?id=zq0CacBD3P>.
 - [5] Zhibin Gou, Zhihong Shao, Yeyun Gong, Yelong Shen, Yujiu Yang, Minlie Huang, Nan Duan, and Weizhu Chen. ToRA: A tool-integrated reasoning agent for mathematical problem solving. In *The Twelfth International Conference on Learning Representations*, 2024. URL <https://openreview.net/forum?id=Ep0TtjVoap>.
 - [6] Lukas Helff, Quentin Delfosse, David Steinmann, Ruben Härle, Hikaru Shindo, Patrick Schramowski, Wolfgang Stammer, Kristian Kersting, and Felix Friedrich. LLMs gaming verifiers: RLVR can lead to reward hacking. *arXiv preprint arXiv:2604.15149*, 2026.
 - [7] Dan Hendrycks, Collin Burns, Saurav Kadavath, Akul Arora, Steven Basart, Eric Tang, Dawn Song, and Jacob Steinhardt. Measuring mathematical problem solving with the MATH dataset. In Joaquin Vanschoren and Sai-Kit Yeung, editors, *Proceedings of the Neural Information Processing Systems Track on Datasets and Benchmarks*, volume 1, 2021. URL <https://datasets-benchmarks-proceedings.neurips.cc/paper/2021/hash/be83ab3ecd0db773eb2dc1b0a17836a1-Abstract-round2.html>.
 - [8] Chengsong Huang, Wenhao Yu, Xiaoyang Wang, Hongming Zhang, Zongxia Li, Ruosen Li, Jiaxin Huang, Haitao Mi, and Dong Yu. R-Zero: Self-evolving reasoning LLM from zero data. In *The Fourteenth International Conference on Learning Representations*, 2026. URL <https://openreview.net/forum?id=96apU6YzS0>.
 - [9] Yiming Huang, Xiao Liu, Yeyun Gong, Zhibin Gou, Yelong Shen, Nan Duan, and Weizhu Chen. Key-point-driven data synthesis with its enhancement on mathematical reasoning. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 39, pages 24176–24184, 2025. doi: 10.1609/aaai.v39i23.34593. URL <https://ojs.aaai.org/index.php/AAAI/article/view/34593>.
 - [10] Thomas Hubert, Rishi Mehta, Laurent Sartran, et al. Olympiad-level formal mathematical reasoning with reinforcement learning. *Nature*, 651:607–613, 2026. doi: 10.1038/s41586-025-09833-y.
 - [11] Hadi Khalaf, Claudio Mayrink Verdun, Alex Oesterling, Himabindu Lakkaraju, and Flavio du Pin Calmon. Inference-time reward hacking in large language models. In *Advances in Neural Information Processing Systems*, 2025. URL <https://openreview.net/forum?id=hSX7Dd8dxy>. Spotlight.
 - [12] Xuefeng Li, Yanheng He, and Pengfei Liu. Synthesizing verified mathematical problems. In *The 4th Workshop on Mathematical Reasoning and AI at NeurIPS 2024*, 2024. URL <https://openreview.net/forum?id=L5US0930w0>.
 - [13] Xiao Liang, Zhong-Zhi Li, Yeyun Gong, Yang Wang, Hengyuan Zhang, Yelong Shen, Ying Nian Wu, and Weizhu Chen. Sws: Self-aware weakness-driven problem synthesis in reinforcement learning for llm reasoning. In *Advances in Neural Information Processing Systems*, 2025. URL <https://openreview.net/forum?id=0jQUNQsZra>.

- [14] Xiao Liang, Zhongzhi Li, Yeyun Gong, Yelong Shen, Ying Nian Wu, Zhijiang Guo, and Weizhu Chen. Beyond pass@1: Self-play with variational problem synthesis sustains rlvr. In *The Fourteenth International Conference on Learning Representations*, 2026. URL <https://openreview.net/forum?id=Wjf30MJxpn>.
- [15] Haipeng Luo, Qingfeng Sun, Can Xu, Pu Zhao, Jian-Guang Lou, Chongyang Tao, Xiubo Geng, Qingwei Lin, Shifeng Chen, Yansong Tang, and Dongmei Zhang. Wizardmath: Empowering mathematical reasoning for large language models via reinforced evol-instruct. In *The Thirteenth International Conference on Learning Representations*, 2025. URL <https://openreview.net/forum?id=mMPMHWOd0y>.
- [16] Chaitanya Manem, Pratik Prabhanjan Brahma, Prakamya Mishra, Zicheng Liu, and Emad Barsoum. Sand-math: Using llms to generate novel, difficult and useful mathematics questions and answers, 2025. URL <https://arxiv.org/abs/2507.20527>.
- [17] Ivan Moshkov, Darragh Hanley, Ivan Sorokin, Shubham Toshniwal, Christof Henkel, Benedikt Schifferer, Wei Du, and Igor Gitman. Aimo-2 winning solution: Building state-of-the-art mathematical reasoning models with openmathreasoning dataset, 2025. URL <https://arxiv.org/abs/2504.16891>.
- [18] OpenAI. Learning to reason with LLMs. <https://openai.com/index/learning-to-reason-with-llms/>, 2024. Accessed: 2026-05-07.
- [19] Arkil Patel, Siva Reddy, and Dzmitry Bahdanau. How to get your LLM to generate challenging problems for evaluation. In *NeurIPS 2025 Workshop on LLM Evaluation*, 2025. URL <https://openreview.net/forum?id=AQm9quyPHU>.
- [20] Qizhi Pei, Lijun Wu, Zhuoshi Pan, Yu Li, Honglin Lin, Chenlin Ming, Xin Gao, Conghui He, and Rui Yan. MathFusion: Enhancing mathematical problem-solving of LLM through instruction fusion. In Wanxiang Che, Joyce Nabende, Ekaterina Shutova, and Mohammad Taher Pilehvar, editors, *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 7400–7420, Vienna, Austria, July 2025. Association for Computational Linguistics. doi: 10.18653/v1/2025.acl-long.367. URL <https://aclanthology.org/2025.acl-long.367/>.
- [21] David Rein, Betty Li Hou, Asa Cooper Stickland, Jackson Petty, Richard Yuanzhe Pang, Julien Dirani, Julian Michael, and Samuel R. Bowman. GPQA: A graduate-level google-proof q&a benchmark. In *First Conference on Language Modeling*, 2024. URL <https://openreview.net/forum?id=Ti67584b98>.
- [22] Zhihong Shao, Peiyi Wang, Qihao Zhu, Runxin Xu, Junxiao Song, Xiao Bi, Haowei Zhang, Mingchuan Zhang, Y. K. Li, Y. Wu, and Daya Guo. Deepseekmath: Pushing the limits of mathematical reasoning in open language models. *arXiv preprint arXiv:2402.03300*, 2024.
- [23] Zhengyang Tang, Xingxing Zhang, Benyou Wang, and Furu Wei. MathScale: Scaling instruction tuning for mathematical reasoning. In Ruslan Salakhutdinov, Zico Kolter, Katherine Heller, Adrian Weller, Nuria Oliver, Jonathan Scarlett, and Felix Berkenkamp, editors, *Proceedings of the 41st International Conference on Machine Learning*, volume 235 of *Proceedings of Machine Learning Research*, pages 47885–47900. PMLR, 21–27 Jul 2024. URL <https://proceedings.mlr.press/v235/tang24k.html>.
- [24] Shubham Toshniwal, Wei Du, Ivan Moshkov, Branislav Kisacanin, Alexan Ayrapetyan, and Igor Gitman. Openmathinstruct-2: Accelerating ai for math with massive open-source instruction data. In *The Thirteenth International Conference on Learning Representations*, 2025. URL <https://openreview.net/forum?id=mTCbq2QssD>.
- [25] Trieu H. Trinh, Yuhuai Wu, Quoc V. Le, He He, and Thang Luong. Solving olympiad geometry without human demonstrations. *Nature*, 625(7995):476–482, 2024. doi: 10.1038/s41586-023-06747-5.
- [26] Xiong Jun Wu, Zhenduo Zhang, Zujie Wen, Zhiqiang Zhang, Wang Ren, Lei Shi, Cai Chen, Deng Zhao, Qing Wang, Xudong Han, Chengfu Tang, Dingnan Jin, Qing Cui, and Jun Zhou. Sharp: Synthesizing high-quality aligned reasoning problems for large reasoning models reinforcement learning, 2025. URL <https://arxiv.org/abs/2505.14147>.

- [27] Longhui Yu, Weisen Jiang, Han Shi, Jincheng YU, Zhengying Liu, Yu Zhang, James Kwok, Zhenguo Li, Adrian Weller, and Weiyang Liu. Metamath: Bootstrap your own mathematical questions for large language models. In *The Twelfth International Conference on Learning Representations*, 2024. URL <https://openreview.net/forum?id=N8N0hgNDRt>.
- [28] Weizhe Yuan, Richard Yuanzhe Pang, Kyunghyun Cho, Xian Li, Sainbayar Sukhbaatar, Jing Xu, and Jason E. Weston. Self-rewarding language models. In Ruslan Salakhutdinov, Zico Kolter, Katherine Heller, Adrian Weller, Nuria Oliver, Jonathan Scarlett, and Felix Berkenkamp, editors, *Proceedings of the 41st International Conference on Machine Learning*, volume 235 of *Proceedings of Machine Learning Research*, pages 57905–57923. PMLR, 21–27 Jul 2024. URL <https://proceedings.mlr.press/v235/yuan24d.html>.
- [29] Shaoxiong Zhan, Yanlin Lai, Ziyu Lu, Dahua Lin, Ziqing Yang, and Fei Tan. Mathsmith: Towards extremely hard mathematical reasoning by forging synthetic problems with a reinforced policy, 2025. URL <https://arxiv.org/abs/2508.05592>.
- [30] Andrew Zhao, Yiran Wu, Yang Yue, Tong Wu, Quentin Xu, Yang Yue, Matthieu Lin, Shenzhi Wang, Qingyun Wu, Zilong Zheng, and Gao Huang. Absolute zero: Reinforced self-play reasoning with zero data. In *Advances in Neural Information Processing Systems*, 2025. URL <https://openreview.net/forum?id=neZSGqhxDa>. Spotlight.
- [31] Xueliang Zhao, Wei Wu, Jian Guan, and Lingpeng Kong. PromptCoT: Synthesizing olympiad-level problems for mathematical reasoning in large language models. In Wanxiang Che, Joyce Nabende, Ekaterina Shutova, and Mohammad Taher Pilehvar, editors, *Findings of the Association for Computational Linguistics: ACL 2025*, pages 18167–18188, Vienna, Austria, July 2025. Association for Computational Linguistics. doi: 10.18653/v1/2025.findings-acl.935. URL <https://aclanthology.org/2025.findings-acl.935/>.

A Implementation Details

This appendix summarizes implementation details for the two experimental regimes.

Software stack. Training uses PyTorch and HuggingFace Transformers with ver1 for supervised finetuning and GRPO-style reinforcement learning. Online rollouts and offline solver evaluations use vLLM. The hard verifier for indefinite integral uses SymPy-backed parsing, differentiation, and expression matching. The general math setting uses rule-based filters, an LLM judge, and the same answer-extraction and correctness-checking stack used for solver evaluation. Unless otherwise noted, trainable models are initialized from Qwen3-4B-Base.

Compute. Training and large-scale generation/evaluation runs use 8 GPUs. A full training/evaluation round takes approximately 60 hours. We report only the hardware scale needed to reproduce the experiments and omit cluster-specific identifiers.

Detailed procedure. Algorithm 1 gives the full VHG procedure described in Section 2. The two experimental regimes share the same ordering; they differ only in the verifier backend and task-specific filters.

Algorithm 1 Verifier-backed Hard Problem Generation (VHG)

Require: Seed pool $\mathcal{S}$; setter Q_θ ; solver S_ϕ ; verifier V ; solver sampling budget K ; per-round generation budget N ; rounds T

- 1: Initialize Q_θ by cold-SFT on a small set of seed-conditioned (x, y^*) examples
- 2: **for** $t = 1, \dots, T$ **do**
- 3: $\mathcal{G}_t \leftarrow \emptyset$
- 4: **for** $i = 1, \dots, N$ **do**
- 5: Sample seed $s \sim \mathcal{S}$ and generate $(x_i, y_i^*) \sim Q_\theta(\cdot \mid s)$
- 6: $v_i \leftarrow V(x_i, y_i^*)$ $\triangleright$ validity gate
- 7: **if** $v_i = 1$ **then**
- 8: Sample $\{\hat{y}_i^{(k)}\}_{k=1}^K \sim S_\phi(\cdot \mid x_i)$ and compute $\text{Acc}_S(x_i, y_i^*)$
- 9: **else**
- 10: $\text{Acc}_S(x_i, y_i^*) \leftarrow 0$ $\triangleright$ not used: gated by v_i below
- 11: **end if**
- 12: $r_i \leftarrow v_i(1 - \text{Acc}_S(x_i, y_i^*))$ $\triangleright$ setter reward
- 13: $\mathcal{G}_t \leftarrow \mathcal{G}_t \cup \{(x_i, y_i^*, v_i, \text{Acc}_S(x_i, y_i^*), r_i)\}$
- 14: **end for**
- 15: Update setter: $\theta \leftarrow \text{RL-update}(\theta, \{(x_i, y_i^*, r_i)\}_{i=1}^N)$
- 16: Build verifier-accepted pool $\mathcal{D}_V^{(t)} \leftarrow \{(x_i, y_i^*) \in \mathcal{G}_t : v_i = 1\}$ with quality filters and deduplication
- 17: Update solver ϕ on $\mathcal{D}_V^{(t)}$ with task-correctness reward $R_S(x, y^*) = \text{Acc}_S(x, y^*)$
- 18: **end for**
- 19: **Output:** trained Q_θ , S_ϕ , and challenge pool ranked by local difficulty $1 - \text{Acc}_S$

Table 4: Indefinite-integral data sizes. “Accepted” denotes acceptance by the hard verifier backend.

Stage	Size	Notes
Cold-SFT setter examples	400	Frontier-model generated examples for initializing the setter; split into 320 training and 80 held-out validation examples.
Parsed, deduplicated generated candidates	18 663	Raw candidate pool before the hard-verifier prefilter and before pass-rate filtering.
Construction-time verifier-retained candidates	4076	Retained by the construction-time verifier prefilter.
Reference-valid candidates	4074	Two retained candidates fail the later reference check.
Generated solver-data component	3939	Non-duplicate accepted pool used as the generated component for solver-data construction.
Challenge pool	1000	Accepted non-duplicate candidates selected by construction-time local solver difficulty.

Table 5: General-math data sizes. Acceptance is judge-based rather than exact verification.

Stage	Size	Notes
Seed prompts for cold-SFT collection	5000	Competition-math seed prompts used for the cold-SFT collection stage.
Judge-validated cold-SFT examples	1892	General-math setter initialization examples, split into 1513 train and 379 validation examples.
Easy seed set for setter RL	1711	Selected from 5000 MATH seed problems using Qwen3-4B-Base Pass@1 ≥ 0.75 ; split into 1368 train and 343 validation examples.
Raw generated candidate outputs	400 000	Generated by the trained setter before local filtering.
Unique candidates after local filtering	270 064	Format, answer, copy, and degeneracy filters applied.
Template-deduplicated candidates	230 532	Deduplicated before pass-rate and judge-validity filtering.
Local-solver filtered candidates	107 398	Candidates retained after construction-time local solver filtering.
Judge-evaluated candidates	46 080	Candidates submitted to the LLM judge for solver-data construction.
Judge-accepted solver-data pool	20 670	Split into 16 536 train and 4134 validation examples.
Challenge-pool selection	2129 $\rightarrow$ 100	Construction-time hard candidates filtered to the final non-duplicate challenge pool.

Table 6: Optimization hyperparameters. These defaults are used unless a run-specific override is supplied.

Stage	Trainer and objective	Learning rate	Epochs, checkpoints, and batch sizes
Setter cold-SFT	Supervised finetuning in <code>ver1</code> on seed-conditioned problem-generation examples.	1×10^{-5}	5 epochs; global batch 32; micro-batch 4; selected checkpoint step 50 for the integration initializer and step 300 for the general-math initializer.
Setter RL	GRPO-style actor training in <code>ver1</code> ; reward is verifier-gated hardness from Eq. 1.	2×10^{-6}	Up to 200 epochs; train batch 128; PPO mini-batch 64; PPO micro-batch 8; selected integration step: 200; general-math pool generation uses checkpoints every 25 steps through step 225.
Solver RL	GRPO-style actor training in <code>ver1</code> ; reward is task correctness on verifier-accepted data.	2×10^{-6}	Up to 100 epochs; train batch 128; PPO mini-batch 64; PPO micro-batch 8; selected integration step: 500; selected general-math step: 500.

Table 7: Rollout and sequence settings.

Stage	Rollouts	Validation samples	Lengths	Sampling setup
Setter cold-SFT	Not applicable.	Held-out validation split from the SFT examples.	Maximum sequence length 4096.	Supervised finetuning with held-out validation.
Setter RL	8 rollouts per prompt.	10 samples per prompt.	Maximum response length 8192; prompt length 4096 for the general-math pipeline.	Rollout temperature 1.0; top- p 1.0; vLLM rollouts.
Solver RL	8 rollouts per prompt.	10 samples per prompt.	Maximum response length 8192; prompt length 4096 for the general-math pipeline.	Validation temperature 1.0; top- p 0.7; vLLM rollouts.

Verifier and filtering settings. For indefinite integral, the setter reward and pool filtering use a hard verifier that rejects unparseable expressions and accepts a pair only when differentiating the candidate antiderivative matches the generated integrand. Construction-time local solver difficulty is estimated with 10 samples per generated problem. The generated component for solver-data construction is the non-duplicate accepted pool, which is then mixed with collected non-synthetic data.

For general math, local filters first reject malformed outputs, missing or multiple boxed answers, near-copies of the seed, degenerate final answers, and high-level formatting failures. Candidate difficulty is estimated with 10 local solver samples using temperature 0.7 and top- p 0.95. The main solver-training pool keeps candidates in the local-pass band $[0.1, 0.9]$ and then applies the LLM judge; the judge uses GPT-5.4 with 512-token responses. The soft-verifier prompt is shown in Appendix E.4.

Evaluation settings. For downstream solver benchmarks, we report Pass@1 as the primary metric and Pass@8 as the shared low-sample diagnostic. Indefinite-integral benchmark evaluations use 64 samples per problem. General math benchmark evaluations use 16 samples per problem, with a 4096-token answer budget for MATH, GSM8K, Minerva, and Olympiad, and an 8192-token budget for AMC and AIME. Challenge-pool evaluations use the same solver prompting and sampling conventions as the corresponding benchmark family.

B Data Curation and Filtering

Held-out integration evaluation-set curation. The Integration Stress Test is curated independently from the generated Exact-Verified training pool. We collect human-authored integration problems from two advanced integration texts and online integration problem pages. Text sources are converted with OCR-assisted extraction, web sources are parsed from embedded math markup, and the resulting records are normalized into integrand/reference-answer pairs using automated cleaning and LLM-assisted extraction. Quality control has four stages: parse the problem into an explicit integrand and variable, verify the reference by differentiating it against the integrand, remove failures such as numeric mismatches or unparsable answers, and stress-test retained references with wrong-answer perturbations that the checker should reject. We then remove normalized overlaps with the seed set used elsewhere in the integration experiments. The final 532-item set is held out from Exact-Verified training and used only for evaluation. It is substantially larger than the AntiderivBench splits used here: 532 problems versus 177 Qualifier and 66 Competition problems.

Generated-pool funnels and challenge filtering. The integration generated-pool analysis starts from 18 663 parsed and deduplicated candidates before verifier prefiltering and pass-rate filtering. The construction-time verifier prefilter retains 4076 candidates, of which 4074 pass the later reference-validity check. Within the verifier-retained generated pool, 1144 problems have zero local pass rate under the local 4B solver, so the challenge subset is drawn from a substantial reservoir of verifier-accepted hard problems rather than from a few outliers. For general math, the largest generated-candidate analysis starts from 400 000 candidate outputs, applies local format, answer, copy, degeneracy, template-deduplication, local-solver, and judge-verifier filters, and yields 16 536 solver-training rows with 4134 held-out rows. The final general math challenge filtering selects 100 non-duplicate challenge problems from 2129 checked construction-time hard candidates, a 4.70% selection rate.

Table 8: Data-quality and verifier-acceptance funnels for the held-out integration evaluation set, the Exact-Verified integration pool, and the Judge-Verified general math training and challenge pools. General math rows are judge-validated construction statistics, not exact-validity guarantees.

Regime	Stage	Count	Diagnostic or role
Integration Stress Test	Collected records	869	OCR- and web-extracted human-authored integration items
Integration Stress Test	Verifier-passed retained pool	575	Clean problem-reference pairs after extraction and checking
Integration Stress Test	Wrong-answer false positives	0	Perturbed incorrect references accepted by the checker
Integration Stress Test	Overlap removals	43	Normalized overlaps removed against the seed set
Integration Stress Test	Final held-out split	532	Evaluation-only split, larger than the AntiderivBench splits used here
Exact-Verified integration pool	Verifier-retained candidates	4076	Construction-time retained candidates before the later reference-validity check
Exact-Verified integration pool	Reference-valid pool	4074	36.1% mean local pass rate; 10.6 mean complexity
Exact-Verified integration pool	Generated solver-data component	3939	33.9% mean local pass rate; 10.8 mean complexity
Exact-Verified integration pool	Challenge subset	1000	0.0% mean local pass rate; 16.4 mean complexity
Judge-Verified general math training	Candidate outputs	400 000	Generated outputs before local filtering
Judge-Verified general math training	Unique problems after local filtering	270 064	Format, answer, copy, and degeneracy filters applied
Judge-Verified general math training	Deduplicated problem templates	230 532	Template-aware duplicate reduction
Judge-Verified general math training	Solver-filtered candidates	107 398	Local solver filtering before judge validation
Judge-Verified general math training	Judge-checked candidates	46 080	Candidates submitted to the judge verifier
Judge-Verified general math training	Judge-accepted pairs	20 670	44.9% judge-validated acceptance rate
Judge-Verified general math training	Training split	16 536	Solver-training rows
Judge-Verified general math training	Held-out split	4134	Held-out generated rows
Judge-Verified general math challenge	Checked hard candidates	2129	Construction-time hard candidates under the 4B solver
Judge-Verified general math challenge	Final non-duplicate challenge subset	100	4.70% selection rate for stronger-solver challenge evaluation

C Additional Experiment Results

General math subgroup results. Table 9 provides the detailed general math subgroup comparison and ablation evidence. The solver trained with VHG (Soft) is best overall and best or tied on most benchmark families, while the vanilla GRPO and SFT-data ablations remain competitive on selected subsets. The table is intended as subgroup robustness evidence rather than a statistical-significance claim.

Table 9: General math subgroup Pass@1 comparison including AIME 2026. Values are percentages over 2896 benchmark problems; bold marks the best value in each column among the rows shown. R-Zero rows follow the authors’ released implementation and evaluation protocol. The VHG (Soft) row reports the solver trained with judge-validated generated data.

Method	MATH	GSM8K	AMC	Olympiad	Minerva	AIME24	AIME25	AIME26	Overall p@1
Qwen3-4B-Base	66.92	73.91	43.28	34.88	27.78	7.29	8.12	7.71	56.79
Vanilla-GRPO	76.76	90.17	52.50	39.60	31.92	13.96	10.83	8.12	67.62
w. SFT data	76.28	89.89	52.97	39.91	32.63	11.04	11.46	9.58	67.54
R-Zero									
Iter. 1	73.51	90.65	52.34	36.16	23.74	10.62	7.08	6.25	65.61
Iter. 2	73.59	91.37	49.84	35.41	28.52	10.00	7.71	7.50	66.20
Iter. 3	72.91	91.49	50.31	34.06	28.12	10.83	4.17	6.67	65.76
VHG (Soft)	78.99	90.61	55.31	42.13	33.32	13.12	11.46	12.92	69.01

Representative challenge examples. Tables 10 and 11 give representative examples of generated challenge questions with zero Pass@1 under Qwen3-4B-Base. The examples illustrate substantive transformations of the seed problems. In the integration examples, the generated integrands introduce product-rule and logarithmic interactions while remaining accepted by the hard verifier. In the general math examples, the generated problems add geometric or vector constraints that require additional reasoning beyond the seed template.

Table 10: Indefinite-integral challenge examples generated by VHG.

Example 1. Seed function:
$3e^{\sqrt[3]{x}}(x^{2/3} - 2\sqrt[3]{x} + 2).$
Generated challenge integral: find an antiderivative of
$e^{\sqrt[3]{x}}x^{-2/3}\log x q(x) + \frac{3e^{\sqrt[3]{x}}}{x}q(x)$
$+ 3e^{\sqrt[3]{x}}\log x \left(\frac{2}{3}x^{-1/3} - \frac{2}{3}x^{-2/3} \right),$
where $q(x) = x^{2/3} - 2\sqrt[3]{x} + 2$.
Example 2. Seed function:
$\frac{3}{2}(x+1)^{2/3} - 3\sqrt[3]{x+1} + 3\log(\sqrt[3]{x+1} + 1).$
Generated challenge integral: find an antiderivative of
$\frac{1}{3}(x+1)^{-2/3}r(x)$
$+ (x+1)^{1/3}\left((x+1)^{-1/3} - (x+1)^{-2/3} + \frac{(x+1)^{-2/3}}{\sqrt[3]{x+1} + 1}\right),$
where $r(x)$ is the seed function.

D Generation Analysis Details

Setter learning diagnostics. Figure 6 and Figure 7 provide validation-set views of the hard-verifier setter trajectory discussed in Section 5. Early checkpoints mainly improve reference validity, while later checkpoints move valid samples toward lower local solver pass rates. Figure 8 gives the corresponding rollout-window view, showing that the mass of valid hard generations grows over training.

Table 11: General-math challenge examples generated by VHG.

Example 1. Original seed: In triangle ABC , two tangent half-angle products are given; find the third product.
Generated challenge problem: In triangle ABC , the same two tangent half-angle products are given and the sides satisfy $a/13 = b/15$. Find $\tan(\frac{A-B}{2}) \tan \frac{C}{2}$.

Example 2. Original seed: The vectors satisfying $\mathbf{v} \cdot \mathbf{v} = \mathbf{v} \cdot \mathbf{c}$, with $\mathbf{c} = (10, -40, 8)$, form a solid; find its volume.

Generated challenge problem: The vectors satisfying $\mathbf{v} \cdot \mathbf{v} = \mathbf{v} \cdot \mathbf{a} + \mathbf{v} \cdot \mathbf{b} + \mathbf{v} \cdot \mathbf{c}$ form a solid, for $\mathbf{a} = (1, 2, 3)$, $\mathbf{b} = (4, -1, 2)$, and $\mathbf{c} = (6, 5, -1)$. Find its volume.

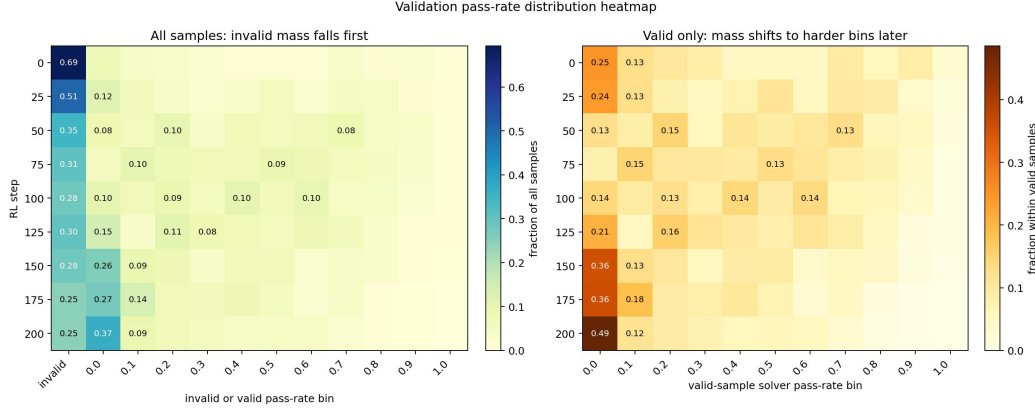


Figure 6: Validation pass-rate heatmap for the hard-verifier setter. Rows correspond to validation checkpoints and columns to local solver pass-rate bins. Accepted validation samples move toward harder bins as training proceeds.

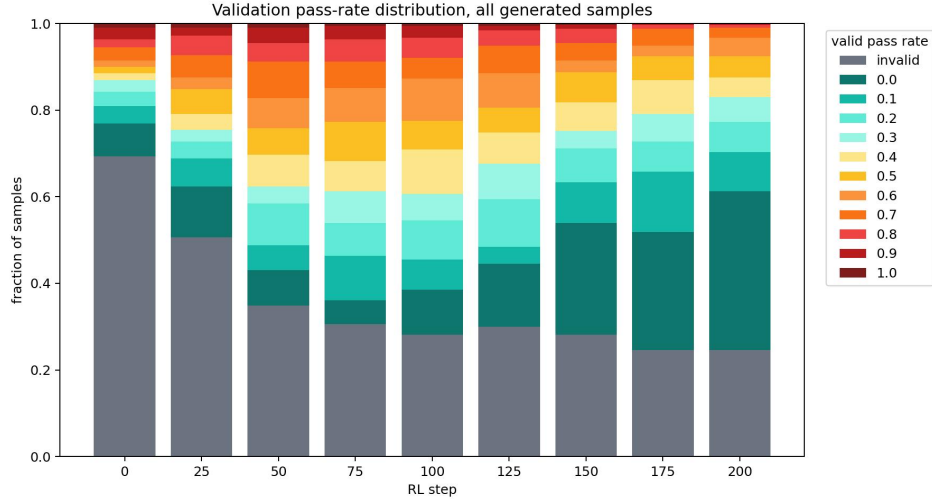


Figure 7: Validation pass-rate distributions for the hard-verifier setter. Later checkpoints contain a larger fraction of valid samples with low local solver pass rate, indicating harder generated problem-reference pairs after validity improves.

General-math hardness-validity analysis. Figure 9 provides the soft-verifier counterpart to the indefinite-integral analysis in Section 5. Because exact checking is unavailable for general math, the validity curve is based on LLM-as-a-judge validation rather than an exact mathematical equivalence test. The figure should therefore be read as evidence that judge-backed filtering can recover valid hard candidates from a noisy generated pool, not as an exact-validity guarantee.

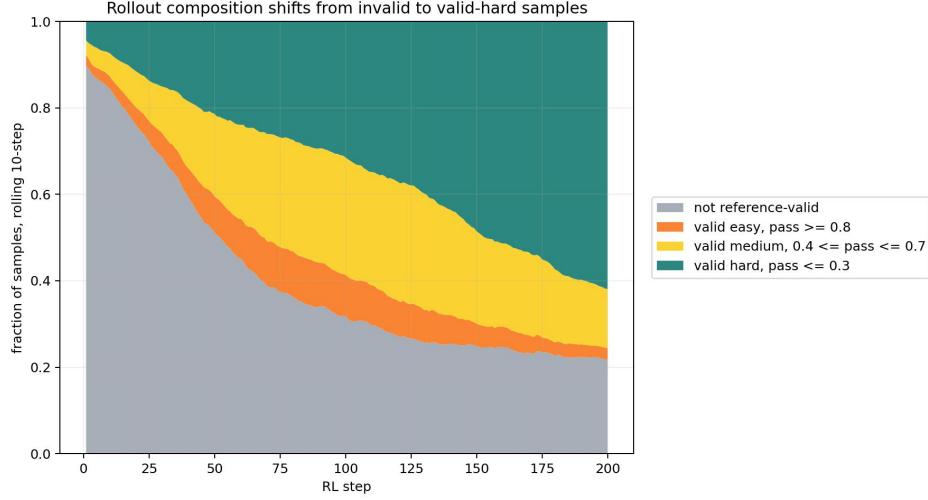


Figure 8: Rollout-window hardness categories for the hard-verifier setter. The share of valid hard candidates increases across rollout windows, consistent with the validation trajectory in Figure 4.

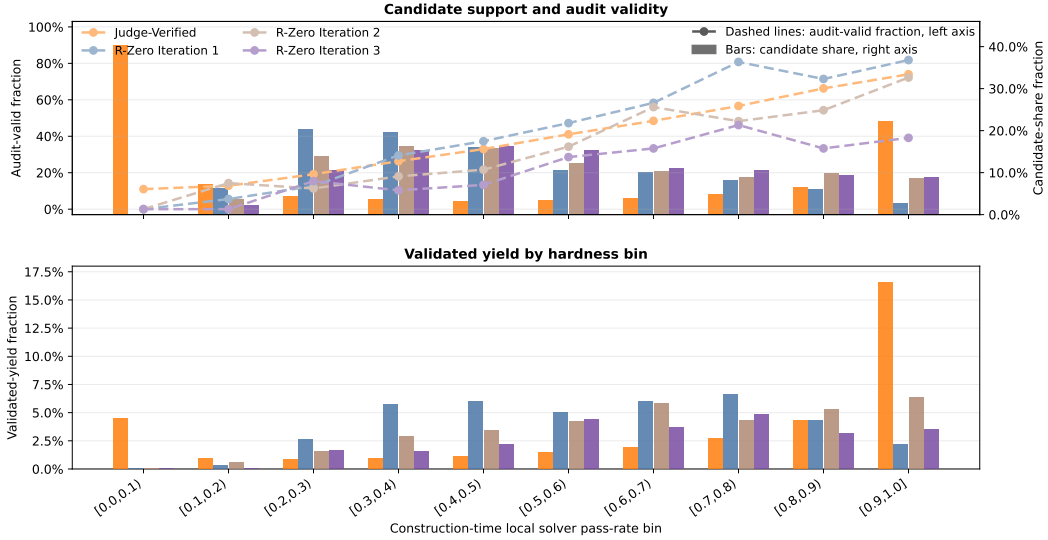


Figure 9: Hardness-validity profile for general math under model-based verification. Candidates are binned by construction-time local solver pass rate. Top: dashed curves show judge-valid fraction within each bin using the left axis, and solid bars show candidate share within each raw generated pool using the right axis. R-Zero points at $[0.0, 0.1]$ are plotted at zero because those iterations have no candidates in the hardest bin. Bottom: bars show validated yield. This is a judge-validity analysis rather than an exact-validity guarantee.

Rollout-level novelty and reuse. We analyze integration generation streams before downstream solver training to test whether the setter copies seed antiderivatives, whether the same generated integrand is reused across different seeds, and whether verifier-matched candidates continue to add novel hard problems over training. The recorded quality analysis covers 200 training rollout steps with 204 800 candidate samples and nine validation checkpoints with 2970 candidate samples. Copying and cross-seed reuse are both low in the integration stream: only 4.6% of parsed generated references copy the seed reference, and only 0.15% of parsed generated integrands appear under multiple seeds. Validation checkpoints show a slightly higher seed-copy rate, 6.6%, but no cross-seed integrand reuse. At the same time, the training stream contributes 9077 new verifier-matched, globally novel problems.

The per-step new sets remain nontrivial for the solver: weighted Pass@1 is 71.1%, and the hardest rollout step has Pass@1 of only 24.5%.

Table 12: Rollout-level quality diagnostics for Exact-Verified. Seed-copy and cross-seed reuse measure direct copying and repeated generated integrands. New matched counts verifier-matched globally novel additions. Hardest Pass@1 is the lowest per-step single-sample solver success among newly added verifier-matched problems.

Stream	Steps	Samples	Seed-copy	Cross-seed reuse	New matched	Matched/local	Novel/matched	Hardest Pass@1
Training rollouts	200	204 800	4.6%	0.15%	9077	38.7%	27.6%	24.5%
Validation checkpoints	9	2970	6.6%	0.00%	253	48.0%	33.9%	22.4%

E Prompts

Prompt-template scope. The production prompts include task-specific formatting instructions and sampling wrappers. For readability, we report normalized paper-facing templates that preserve the information available to each component and the acceptance criteria used by the verifier.

E.1 Setter Generation Prompt

The setter receives a seed problem-reference pair and generates a related but nontrivial new pair. The same high-level template is used in both regimes, with task-specific instructions for indefinite integral or general math:

Instruction. Given a seed problem and its reference solution, create a new mathematical problem that is related to the seed but is not a copy or cosmetic rewrite. Also provide a complete reference solution and a final answer.

Input fields. Seed Problem; Seed Reference Solution.

Requirements. The generated problem must be well-posed, mathematically meaningful, and sufficiently specified. The generated reference solution must solve the generated problem, and the final answer must be explicit. The generated pair should preserve a recognizable relation to the seed while introducing a substantive mathematical modification.

Output fields. Generated Problem; Generated Reference Solution; Final Answer.

E.2 Solver Prompt

The solver receives only the generated problem, not the seed or the verifier decision:

Instruction. Solve the following mathematics problem. Provide a concise derivation and put the final answer in a boxed expression.

Input field. Problem.

Output requirement. The final response must contain exactly one final answer that can be extracted for correctness checking.

E.3 Hard-Verifier Input Format

The indefinite-integral verifier is not an LLM prompt. It receives a generated integrand, variable, and candidate antiderivative, and accepts the pair only when the parsed derivative of the candidate antiderivative matches the generated integrand under the checker. The paper-facing interface is:

Input fields. Integration variable; Generated Integrand; Candidate Antiderivative.

Acceptance criteria. The expressions must parse successfully, the variable must be unambiguous, and differentiating the candidate antiderivative with respect to the variable must match the generated integrand after checker normalization. Degenerate, unparsable, or ambiguous pairs are rejected.

E.4 Soft-Verifier Prompt

The general math verifier receives the seed problem, the seed solution, the generated problem, and the generated solution. The prompt asks the LLM judge to evaluate the generated pair relative to the seed, rather than judging the generated solution in isolation. The paper-facing template is:

Instruction. You are a careful math verifier. Judge the generated pair relative to the seed. Judge only validity and relation to the seed; do not require the variant to be harder.

Input fields. Seed Problem; Seed Solution; Derived Problem; Modified Solution.

Acceptance criteria. The generated problem must be mathematically well-posed, unambiguous, and sufficiently specified to determine the requested answer. The generated solution must correctly solve the generated problem, and the final boxed answer must match the solution and fully answer every requested quantity. Degenerate variants whose intended final answer is no solution, empty set, undefined, impossible, or inconsistent are rejected. The generated problem must remain recognizably related to the seed rather than jumping to an unrelated topic or method. Exact copies, near copies, cosmetic paraphrases, and variable renamings with no meaningful mathematical change are rejected.

Required response fields. The judge first gives one short paragraph explaining the check, then returns Boolean tags for `valid_problem`, `valid_solution`, `seed_anchored`, `not_trivial_copy`, and `complete_final_answer`.

The verifier accepts a generated pair only when the structural filters pass and all required judge fields are true.

F Limitations and Broader Impact

The main limitation of VHG is that its guarantees are only as strong as the verifier backend. The indefinite-integral setting provides a clean hard-verifier testbed, but it is a narrow domain. The general math setting is broader, but its LLM-as-a-judge verifier is softer and can still accept subtle errors, underspecified problems, or reward-hacking artifacts. Thus, our exact-validity claims apply to the hard-verifier setting, while the general math results should be interpreted as evidence for a practical soft-verifier pipeline rather than exact mathematical guarantees.

The empirical comparisons also have finite scope. The consensus baseline is implemented through a representative R-Zero pipeline, but not every comparison perfectly matches generation budget, data mixture, selection rule, or training schedule. Most experiments also use one model family. Broader verifier backends, mathematical domains, and model families would strengthen the evidence for generality.

The broader impact is mixed. Verifier-backed generation can reduce noisy synthetic supervision and make difficulty-seeking data generation easier to inspect. At the same time, automated hard-problem generation can accelerate benchmark overfitting or create misleading stress tests if verifier assumptions are not documented. We therefore view explicit verifier documentation, separation of hard- and soft-verifier claims, and additional independent validation as important safeguards.